DevOps Containerization Report

DevOps, Containerization & Production Deployment Report

Work Credit RAG Platform — H200 Production Server

Repository: `Work_RAG-Server-Setup` (GitHub: `AliNikkhah2001/Work_RAG-Server-Setup`)
Branch: `main`
Parent Repo: `Work_Credit-RAG_Phase1` (GitHub: `AliNikkhah2001/Work_Credit-RAG_Phase1`)
Agent-State Repo: `ICS_ChatBot_agent-state` (GitHub: `AliNikkhah2001/ICS_ChatBot_agent-state`)
Target Host: 2× NVIDIA H200 NVL (ai-gpu1), Docker available, Self-hosted Jenkins as root
Container Registry: Local H200 Docker registry (`local-h200:5000`)
Corporate Proxy: `http://192.168.203.2:3128` (Squid — all outbound traffic)
Date: 2026-10-31
Status: Phase 1 complete (all files committed & pushed)
Phase 2 pending: Build images on H200, alt-port validation, E2E test, production cutover

---

Table of Contents

1. Executive Summary
2. System Architecture Diagrams
3. Production Service Inventory
4. Port Map
5. Docker Images & Container Specifications

6. CI/CD — Self-Hosted Jenkins
7. GitHub Branch Protection Rules
8. Model Serving — Triton / TensorRT-LLM Optimization
9. Monitoring & Observability
10. Integrations — Inter-Service Communication
11. Scaling Considerations
12. Offline Provisioning Behind Corporate Proxy
13. Containerization Migration Plan — Zero-Downtime
14. Rollback Plan
15. File Deliverables & Locations
16. Acceptance Criteria
17. Known Gotchas & Risks

---

1. Executive Summary

The Work Credit RAG Platform runs on a 2× H200 NVL host (ai-gpu1) with 7 production services currently running as bare host processes (Python venvs) or individual Docker containers. This report details the complete DevOps, containerization, CI/CD, model serving, monitoring, and migration plan to containerize all services **without breaking any running process**.

Key principle: Zero services are stopped or modified during build and validation phases (1–3). Production traffic is unaffected until per-service cutover (Phase 4), with rollback capability for each service.

---

2. System Architecture Diagrams

2.1 High-Level Topology

```
graph TB
    subgraph HOST["H200 Host — 2× H200 NVL (143 GiB VRAM each)"]
        subgraph CONTAINERS["Docker Containers (Production)"]
            KB[KB Retrieval :8000<br/>pgvector + BM25 + RRF + Rerank]
            GUARD[Guardrails :8200<br/>Persian policy + NeMo judge]
            ORCH[Orchestrator :8100<br/>LangGraph RAG pipeline]
            TRACE[Tracing :3000<br/>Fallback Langfuse collector]
            MG[LLM Manager :9000<br/>OpenAI-compatible gateway<br/>11-model registry]
            G1[Gemma ×5 :8080-8084<br/>llama.cpp backends]
            EMB1[Embed E5-small :8001<br/>384-d]
            EMB2[Embed BGE-M3 :8002<br/>1024-d]
            EMB3[Embed MiniLM :8003<br/>384-d]
            WUI[Open WebUI :13000<br/>Frontend chat UI]
```

```
        end
        subgraph DATA["Data Plane (Docker)"]
            REDIS[(Redis :16379<br/>Cache & queue)]
            QDR[(Qdrant :16333<br/>Vector store)]
            PG[(pgvector :15432<br/>Postgres + vector)]
            MIL[(Milvus :19530<br/>Vector store)]
        end
        subgraph MON["Monitoring (Docker)"]
            PROM[Prometheus :19090<br/>Metrics scraping]
            GRAF[Grafana :13001<br/>Dashboards]
            OTEL[OTel :14317/14318<br/>OTLP + Prometheus export]
            GPU_EXP[GPU Metrics Exporter :9101<br/>nvidia-smi
metrics]
        end
    end

    MG --> G1
    MG --> EMB1
    MG --> EMB2
    MG --> EMB3
    WUI --> MG
    ORCH --> GUARD
    GUARD --> MG
    KB --> EMB1
    KB --> EMB2
    KB --> EMB3
    GRAF --> PROM
    PROM --> GPU_EXP
```

2.2 Request Flow (End-to-End)

```
sequenceDiagram
    participant User
    participant WUI as Open WebUI :13000
    participant ORCH as Orchestrator :8100
    participant GUARD as Guardrails :8200
    participant KB as KB Retrieval :8000
    participant MG as LLM Manager :9000
    participant GEMMA as Gemma ×5 :8080-8084
    participant TRACE as Tracing :3000

    User->>WUI: Send question (Persian)
    WUI->>ORCH: POST /v1/chat/completions
    ORCH->>GUARD: POST /v1/rails/check (input)
    GUARD-->>ORCH: Allowed / Blocked
    alt Blocked
        ORCH-->>WUI: Refusal response
    else Allowed
        ORCH->>KB: POST /search/api
        KB-->>ORCH: Chunks + citations
        ORCH->>GUARD: POST /v1/chat/completions (guarded)
```

```
        GUARD->>MG: Forward to gemma-4-31b
        MG->>GEMMA: Round-robin llama.cpp
        GEMMA-->>MG: Generated text
        MG-->>GUARD: Response
        GUARD-->>ORCH: Output rails applied
        ORCH-->>WUI: Format + citations
    end
    WUI-->>User: Final response with citations
    ORCH->>TRACE: Async trace (JSONL / Langfuse)
```

2.3 CI/CD Pipeline Flow

```
graph LR
    GH[GitHub<br/>Push/PR to main/develop]
    WEB[Webhook<br/>http://<h200>:8080/github-webhook/]
    JEN[Jenkins<br/>Self-hosted on H200<br/>:8080]
    LINT[Lint: ruff + hadolint]
    TEST[Test: pytest + docs-toc]
    BUILD[Build: docker build<br/>7 images]
    PUSH[Push: local-h200:5000]
    DEV[Deploy-dev: alt ports<br/>docker-compose.test.yml]
    E2E[E2E Test: Open WebUI<br/>full chain]
    PROD[Deploy-prod: manual approval<br/>docker-compose.prod.yml]

    GH --> WEB
    WEB --> JEN
    JEN --> LINT
    LINT --> TEST
    TEST --> BUILD
    BUILD --> PUSH
    PUSH --> DEV
    DEV --> E2E
    E2E --> PROD
```

---

3. Production Service Inventory

#	Service	Port	Process Type	Entry Point
1	Gemma backend ×5	8080–8084	host venv + `gemma_supervisor.sh`	`scripts/services/llama_chat_server.py`
2	LLM Inference Manager	9000	host venv	`llm_inference_manager/app.py`
3	Embedding ×3	8001–8003	host venv	`scripts/services/embed_server.py`
4	KB Retrieval	8000	host venv	`kb_manager/web/app.py` (submodul

#	Service	Port	Process Type	Entry Point
5	Guardrails	8200	host venv	`work_rag_guardrails.api:create_ap` (submodule)
6	Orchestrator (LangGraph)	8100	host venv	`work_rag_orchestrator.api:create_` (submodule)
7	Tracing fallback	3000	host venv	`components/tracing/app.py`
8	Open WebUI	13000	Docker (existing)	`ghcr.io/open-webui/open-webui:mai`
9	Redis cache	16379	Docker (existing)	`redis:7-alpine`
10	postgres + pgvector	5432	system cluster	system install
11	Gemma TRT-LLM	8010	host venv	`trtllm-serve`

4. Port Map

Port(s)	Service	Production?	Containerized?	Notes
8000	KB Retrieval	✅ Yes	⏳ Planned	pgvecto HNSW BM25 + RRF + rerank
8001–8003	Embedding ×3	✅ Yes	⏳ Planned	e5-sma bge-m3 MiniLM
8080–8084	Gemma-4-31B ×5	✅ Yes	⏳ Planned (GPU)	llama.c backen GPU sp
8090	Qwen2.5-7B	✅ Yes	⏳ Planned	Single backen
9000	LLM Inference Manager	✅ Yes	⏳ Planned	OpenAI compat gatewa
9001	DeepSeek-V4-Flash	✅ Yes	⏳ Planned	Wrappe service
9621	LightRAG	⚠ Optional	⏳ Planned	Optiona service
13000	Open WebUI	✅ Yes	✅ Existing	Fronter chat UI
15432	pgvector	✅ Yes	✅ Existing	Postgre vector
16333/16334	Qdrant	✅ Yes	✅ Existing	

Port(s)	Service	Production?	Containerized?	Notes
				Vector store
16379	Redis	✅ Yes	✅ Existing	Cache & queue
19530	Milvus	✅ Yes	✅ Existing	Vector store
13001	Grafana	✅ Yes	✅ Existing	Dashbo
19090	Prometheus	✅ Yes	✅ Existing	Metrics scrapin
19100	node-exporter	✅ Yes	✅ Existing	Node metrics
14317/14318/19092	OTel Collector	✅ Yes	✅ Existing	OTLP + Promet export
8010	Gemma TRT-LLM BF16	✅ Yes	⚠ Host process	Champi model
8011	Gemma TRT-LLM FP8-KV	⚠ Validated	⚠ Host process	Validate variant
18000	Gemma llama-server	⚠ Vast only	❌ No	Vast.ai loopbac only

---

5. Docker Images & Container Specifications

5.1 Dockerfile Summary

Dockerfile	Service	Base Image	GPU	Port	Volume Mounts
`Dockerfile.kb`	KB Retrieval	`python:3.11-slim`	No	8000	`/data`, `/models` (ro)
`Dockerfile.guardrails`	Guardrails	`python:3.11-slim`	No	8200	`/app/policies` (ro), `/app/kb` (ro)
`Dockerfile.orchestrator`	LangGraph Orchestrator	`python:3.11-slim`	No	8100	None
`Dockerfile.tracing`	Fallback Collector		No	3000	`/tmp`

Dockerfile	Service	Base Image	GPU	Port	Volume Mounts
		`python:3.11-slim`			
`Dockerfile.manager`	LLM Manager	`python:3.11-slim`	No	9000	`/data`, `/logs`
`Dockerfile.gemma`	Gemma ×5 Backends	`nvidia/cuda:12.4.1-runtime`	**Yes**	8080–8084	`/models` (ro)
`Dockerfile.embed`	Embedding ×3	`python:3.11-slim`	No	8001–8003	`/models` (ro)

5.2 Common Dockerfile Features

All Dockerfiles include: - **Proxy build args** (`HTTP_PROXY`, `HTTPS_PROXY`, `NO_PROXY`) for corporate Squid `192.168.203.2:3128` - **Trusted-host flags** for pip (proxy modifies package contents) - **Non-root user** (`raguser`, uid 1000) - **Health check** endpoint (`/health` or `/api/public/health`) - **CPU-only torch** for KB and Embed services (avoids 500MB+ CUDA toolkit downloads) - **CUDA base image** for Gemma service (GPU required)

5.3 Docker Compose Files

File	Purpose	Ports	Build Context
`docker-compose.test.yml`	Alt-port validation (zero production impact)	8100, 8201, 8101, 3001, 9001, 8085–8089, 8004–8006	Cross-repo
`docker-compose.prod.yml`	Production cutover (same ports as host)	8000, 8200, 8100, 3000, 9000, 8080–8084, 8001–8003	Cross-repo

5.4 Build Script

`deploy/docker/build-all.sh` — One-command build for all 7 images with proxy support:

```
bash Work_RAG-Server-Setup/deploy/docker/build-all.sh local-h200:5000
```

---

6. CI/CD — Self-Hosted Jenkins

6.1 Jenkins Installation (H200, root user)

```
# Run Jenkins as Docker container with Docker socket access
docker run -d -u root -p 8080:8080 -p 50000:50000 \
  -v /var/lib/jenkins:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --name jenkins \
  jenkins/jenkins:lts

# Access: http://<h200-host>:8080
# Initial admin password: docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Required Jenkins plugins: - Docker Pipeline - GitHub - Pipeline - SSH Agent - HTML Publisher (for benchmark reports)

6.2 Jenkins Credentials

Credential ID	Purpose
`github-pat`	GitHub Personal Access Token (clone repos)
`local-registry-cred`	Local H200 Docker registry auth
`h200-ssh-key`	SSH key for H200 deploy (if Jenkins runs elsewhere)

6.3 Pipeline Jobs

Repo	Jenkinsfile Location	Branches
`Work_Credit-RAG_Phase1`	`Jenkinsfile`	`main`, `develop`
`Work_RAG-Server-Setup`	`Jenkinsfile`	`main`

6.4 Pipeline Stages

```
Checkout → Lint (ruff/hadolint) → Test (pytest) → Build Docker
Images → Push to Local Registry → Deploy-Dev (alt ports) → E2E
Test (Open WebUI) → Deploy-Prod (manual approval)
```

6.5 GitHub Webhook Integration

- **Webhook URL:** `http://<h200-host>:8080/github-webhook/`
- **Trigger:** Push/PR to `main`/`develop` on any repo
- **Action:** Jenkins pipeline runs automatically

---

7. GitHub Branch Protection Rules

Rulesets configured (Settings → Rules → Rulesets) on `main`/`master` and `develop` for all 5 repos:

Setting	Value
Require a pull request before merging	☑ Enabled
Required approving reviews	1
Dismiss stale approvals when new commits are pushed	☑ Checked
Require review from Code Owners	☑ Checked (optional)
Require status checks to pass before merging	☑ Checked (CI: lint, test, docker-build)
Require branches to be up to date before merging	☑ Checked
Require conversation resolution before merging	☑ Checked
Allow force pushes	☐ Disabled
Allow branch deletions	☐ Disabled
Include administrators	☑ Checked

Repositories: `Work_Credit-RAG_Phase1`, `Work_RAG-KB`, `Work_RAG-Orchestrator`, `Work_RAG-Guardrails`, `ICS_ChatBot_agent-state`

---

8. Model Serving — Triton / TensorRT-LLM Optimization

8.1 Gemma-4-31B Optimization on H200

The champion `gemma-4-31b` model is served by **TensorRT-LLM 1.3.0rc29** (`trtllm-serve`, OpenAI-compatible `:8010`) and is the ground-truth generator behind the RAG agent.

8.2 Why LLM Inference Is Memory-Bandwidth Bound

Generating one token requires reading the entire set of model weights from VRAM (~62 GiB in BF16 for Gemma-4-31B text). On H200 NVL (~3.9 TB/s memory bandwidth) the single-stream ceiling is roughly 60 tok/s. Throughput only scales above that by **continuous batching**: many sequences share one weight read, amortizing the weight bandwidth cost across the batch.

8.3 Optimization Mechanisms

Mechanism	What It Does	Effect
Continuous batching / iteration-level scheduler	New requests appended as others finish; no head-of-line blocking	Flat ~58 tok/s → multi-thousand tok/s aggregate
Paged KV cache	KV stored in reusable blocks; no fragmentation, prefix reuse	Stable long-context (8192) at high concurrency
CUDA graphs	Decode step captured for fixed batch sizes; removes CPU launch overhead	Fast path for graph-sized batches; off-graph ~4× slower
FlashInfer attention	Fused attention kernels supporting arbitrary head dims	Gemma-4 head_dim=512 MMHA assert bypassed
FP8 KV cache	KV elements in 1 byte instead of 2	~4 GiB VRAM saved; +15% throughput at conc-128
W8A8 weight FP8 (ModelOpt)	Weight elements in 1 byte instead of 2	~8.6 GiB VRAM saved;

Mechanism	What It Does	Effect
		+34–63% throughput
TP = 1	Single H200 holds whole model	No inter-GPU communication overhead

8.4 Blocker Fixes for TRT-LLM Serving

1. **MPI_Comm_spawn deadlock** — Fixed by launching under `mpirun -n 1` + `trtllm-llmapi-launch`
2. **Bare `python3` in launcher** — Fixed by putting venv first on `PATH`
3. **FlashInfer JIT nvcc too old** — Fixed by composing CUDA 13.2 toolkit from pip packages
4. **gemma-4 `head_dim=512` MMHA assert** — Fixed by `attn_backend: FLASHINFER`

8.5 Benchmark Results (Same-Run A/B)

Concurrency	GGUF llama.cpp `:18000`	BF16 TRT-LLM `:8010`	FP8-KV `:8011`	Weight-FP8 `:8011`
1	57.6	52.5	49.9 (0.95×)	77.6 (1.48×)
2	—	101.9	97.1 (0.95×)	150.5 (1.48×)
4	59.1	202.3	193.1 (0.95×)	295.7 (1.46×)
8	59.4	399.3	381.8 (0.96×)	581.9 (1.46×)
16	—	770.8	748.9 (0.97×)	1131.7 (1.46×)
32	—	1375.1	1370.0 (1.00×)	1948.9 (1.39×)
64	—	2357.7	2363.9 (1.00×)	3212.1 (1.34×)
128	—	3194.9	3658.6 (**1.15×**)	4851.7 (**1.63×**)

Single-stream: BF16 ~52 tok/s vs GGUF ~58 tok/s (near parity — GGUF is Q4 quantized, TRT is BF16).
Continuous batching: ~55×–83× aggregate speedup over GGUF at high concurrency.

8.6 Current Deployment

Variant	Port	VRAM	Status
BF16 conc-128 champion	`:8010`	~137.9 GiB / 143.7 GiB	□ Production (ground truth)
FP8-KV	`:8011`	~134 GiB	⚠ Validated, not promoted
Weight-FP8	`:8011`	~129 GiB	⚠ Quality-regressed, not promoted

---

9. Monitoring & Observability

9.1 Stack Overview

Service	Port	Purpose
Prometheus	19090	Scrapes GPU exporter, service metrics, 15s interval
Grafana	13001	Dashboards, auto-provisioned Prometheus datasource
OTel Collector	14317/14318/19092	OTLP gRPC/HTTP + prometheus export
GPU Metrics Exporter	9101	Pure-stdlib Prometheus metrics from `nvidia-smi`

9.2 What Gets Monitored

- **GPU:** VRAM usage, utilization, temperature, power draw (per H200)
- **Services:** `/health` endpoint status, uptime, request latency
- **Models:** tokens/second, time-to-first-token, batch size, KV cache utilization
- **Containers:** CPU, memory, disk, network I/O

9.3 Alerting (Alertmanager)

Alert	Condition	Severity
Service down	`/health` fails	Critical
High VRAM usage	>90%	Warning
High latency	p95 > 5s	Warning
Container restart loop	>3 restarts/min	Critical

9.4 Logging

- Centralized JSONL trace collection (fallback: `/tmp/langfuse_traces.jsonl`)
- Langfuse v2 UI at `:3001` (full trace UI, Postgres+Clickhouse)
- Fallback collector at `:3000` (JSONL timelines, local Studio debugger)

---

10. Integrations — Inter-Service Communication

10.1 Request Chain

```
Browser :13000 → Open WebUI → Orchestrator :8100
  → Guardrails :8200 (input rails)
  → KB :8000 (retrieval)
  → Guardrails :8200 (guarded generate)
  → Gemma :18000 (llama-server) OR :9000 (Manager → 8080-8084)
  → Guardrails output rails
  → Orchestrator format_response → Open WebUI → Browser
  → Async trace → Langfuse :3001 / fallback :3000
```

10.2 Service Dependencies

Service	Depends On	Communication Protocol
KB	Redis, pgvector	HTTP `/search/api`
Guardrails	LLM Manager (:9000)	HTTP `/v1/rails/check`, `/v1/chat/completions`
Orchestrator	KB (:8000), Guardrails (:8200)	HTTP `/search/api`, `/v1/chat/completions`
Tracing	None (standalone)	HTTP `/api/public/ingestion`
LLM Manager		HTTP `/v1/chat/completions` (round-robin)

Service	Depends On	Communication Protocol
	Gemma backends (8080–8084)	
Gemma	None (standalone per-backend)	HTTP `/v1/chat/completions`
Embed	None (standalone per-model)	HTTP `/v1/embeddings`
Open WebUI	Orchestrator (:8100)	HTTP `/v1/chat/completions`

10.3 Environment Variable Integration

Service	Key Env Vars	Default Values
KB	`KB_DB_URL`, `KB_REDIS_URL`, `KB_REDIS_TTL`, `PORT`	`postgresql+asyncpg://postgres:postgres@127.0.0.1:5432/kb_manager`, `redis://127.0.0.1:16379/0`, `600`, `8000`
Guardrails	`UPSTREAM_LLM_BASE_URL`, `UPSTREAM_LLM_MODEL`, `GUARDRAILS_HOST`, `GUARDRAILS_PORT`, `PORT`	`http://127.0.0.1:9000/v1`, `gemma-4-31b`, `0.0.0.0`, `8200`, `8200`
Orchestrator	`KB_BASE_URL`, `GUARDRAILS_BASE_URL`, `LANGFUSE_HOST`, `PORT`	`http://127.0.0.1:8000`, `http://127.0.0.1:8200`, `http://127.0.0.1:3000`, `8100`
Tracing	`PORT`, `TRACES_FILE`	`3000`, `/tmp/langfuse_traces.jsonl`
Manager	`PORT`, `MANAGER_DB_PATH`	`9000`, `/data/manager.db`
Gemma	`MODEL_PATH`, `MODEL_ID`, `PORT`, `N_CTX`, `N_GPU_LAYERS`, `CUDA_VISIBLE_DEVICES`	`/models/gemma-4-31b.gguf`, `gemma-4-31b`, `8080`, `8192`, `-1`, `0`
Embed	`MODEL_PATH`, `MODEL_ID`, `PORT`	`/models`, `multilingual-e5-small`, `8001`

11. Scaling Considerations

11.1 GPU Scaling

Scenario	Approach	Impact
	TP=1, max_batch_size=128, kv_cache_free_gpu_memory_fraction=0.85	~137.9 GiB /

Scenario	Approach	Impact
Single H200 at capacity		143.7 GiB VRAM
Add second H200	Tensor parallelism TP=2 across GPUs	Double model capacity, inter-GPU overhead
Multiple models	CUDA_VISIBLE_DEVICES split per container	Isolated VRAM per model
Gemma ×5 backends	GPU0→8080/8082/8084, GPU1→8081/8083	Load balanced via Manager round-robin

11.2 Horizontal Scaling

Component	Scaling Method	Limit
KB Retrieval	Multiple container replicas behind load balancer	pgvector connection pool
Guardrails	Multiple container replicas	Stateless, scales linearly
Orchestrator	Multiple container replicas	LangGraph state (MemorySaver) limits
Gemma backends	Add more llama.cpp containers	VRAM per H200 (143 GiB)
Embed services	Multiple container replicas	CPU-bound, scales with cores

11.3 VRAM Budget (Single H200)

Component	VRAM Usage
Gemma-4-31B BF16 weights	~62 GiB
KV cache (conc-128, seq-8192)	~76 GiB

Component	VRAM Usage
Total at conc-128	~137.9 GiB / 143.7 GiB
Headroom	~5.8 GiB (4%)

11.4 Bottlenecks

Bottleneck	Mitigation
Memory bandwidth (single-stream)	Continuous batching
VRAM capacity at conc-128	FP8 KV cache, weight FP8 (needs recalibration)
FlashInfer prefill workspace (batch-256)	Cap concurrency at 128
Power capping (~160s sustain)	Expect ~2.9k tok/s sustained vs ~3.8k burst
Proxy-induced pip hash failures	Build on H200 with configured Docker daemon

12. Offline Provisioning Behind Corporate Proxy

12.1 Proxy Configuration

```
        export http_proxy=http://192.168.203.2:3128 https_proxy=http://
192.168.203.2:3128
        source proxy_setup.sh
```

All traffic via Squid `192.168.203.2:3128`: - apt, git, docker daemon, pip, HF Hub - Docker daemon also proxied

12.2 Offline Preparation CLI

Script	Purpose
`start.sh`	Bootstraps Ubuntu: apt deps, Docker (+ daemon proxy), venv, launches CLI in tmux
`proxy_setup.sh`	Persists Squid proxy env + apt + git + docker daemon config
`fix_env.sh`	Rebuilds broken `prep_venv` (python3.12 + huggingface-hub + transformers)
`offline_prepare_cli.py`	

Script	Purpose
	Self-healing downloader/ installer: state machine + retry queue

Downloads: Docker images (`.tar`), Python wheels (`.whl`), HF models (17 repos), sample projects (Dify, AnythingLLM, RAGFlow, LightRAG)

Layout: `offline-prep/{docker-images/, python-packages/, models/huggingface/, sample-projects/, logs/}`

12.3 Critical Gotchas

- `offline-prep/venv/bin/python3.12 -m pip` — **never bare `pip`** (shebangs point at dead `/ai-gpu1/...`).
- `.state.json` / `.retry_queue.json` are stale — treat as history.
- `numpy`/`scipy` conflict → pin `scipy==1.13.1`.
- Localhost calls must bypass Squid → `trust_env=False`.
- Docker builds **must run on H200** where Docker daemon is configured with Squid proxy — proxy modifies package contents during download, causing pip hash verification failures on other hosts.

---

13. Containerization Migration Plan — Zero-Downtime

Phase 1: Image Build COMPLETE

All 7 Dockerfiles built and committed. Build script: `deploy/docker/build-all.sh`

Status: Files pushed to GitHub. Build not yet executed (must run on H200 due to proxy constraints).

Phase 2: Alt-Port Validation (Next)

Run containers on alternate ports without touching production:

```
        docker compose -f Work_RAG-Server-Setup/deploy/docker/docker-compose.test.yml up -d
```

Alt ports: 8100, 8201, 8101, 3001, 9001, 8085–8089, 8004–8006

Phase 3: E2E Test via Open WebUI (Alt Ports)

Verify full chain: WebUI → Orchestrator (:8101) → Guardrails (:8201) → KB (:8100) → Gemma (:8085) → response

Test queries: SBD1, MTX1, shareholders (expect correct answers with citations); generic query (expect clarification)

Phase 4: Per-Service Cutover (Production Ports)

Order	Service	Port	Rollback
1	Tracing	3000	Restart host venv
2	Embed ×3	8001–8003	Restart host venvs
3	KB Retrieval	8000	Restart host venv
4	Guardrails	8200	Restart host venv
5	Orchestrator	8100	Restart host venv
6	LLM Manager	9000	Restart host venv
7	Gemma ×5	8080–8084	Restart `gemma_supervisor.sh`

Max downtime per service: ~30 seconds. Host venvs kept intact for 24h rollback backup.

Phase 5: Decommission Host Venvs

After 24h stability, stop host venv processes and remove (keep as backup for 1 week).

---

14. Rollback Plan

If a container fails health check after cutover to production port:

```
        # Example: KB service on :8000
        docker stop rag-kb && docker rm rag-kb

        # Host venv still intact — restart it:
        cd /splunk-data/v1/Work_Credit-RAG_Phase1/components/
knowledgebase/kb-manager
        source /tmp/kb-venv/bin/activate
        nohup python -m uvicorn kb_manager.web.app:app --host 0.0.0.0 --
port 8000 &
```

Maximum downtime per service: ~30 seconds. No full-system downtime.

---

15. File Deliverables & Locations

In `Work_RAG-Server-Setup` (main branch):

File	Location	Purpose
`README.md`	Root	Full DevOps & operations reference
`README.pdf`	Root	PDF version of README
`Jenkinsfile`	Root	Self-hosted Jenkins CI/CD pipeline
`docs/migration/containerization-report.md`	`docs/migration/`	Full zero-downtime migration plan
`deploy/docker/Dockerfile.kb`	`deploy/docker/`	KB Retrieval container
`deploy/docker/Dockerfile.guardrails`	`deploy/docker/`	Guardrails container
`deploy/docker/Dockerfile.orchestrator`	`deploy/docker/`	LangGraph container
`deploy/docker/Dockerfile.tracing`	`deploy/docker/`	Tracing collector container
`deploy/docker/Dockerfile.manager`	`deploy/docker/`	LLM Manager container
`deploy/docker/Dockerfile.gemma`	`deploy/docker/`	Gemma ×5 GPU container
`deploy/docker/Dockerfile.embed`	`deploy/docker/`	Embed ×3 container
`deploy/docker/build-all.sh`	`deploy/docker/`	One-command image build
`deploy/docker/docker-compose.test.yml`	`deploy/docker/`	Alt-port validation compose
`deploy/docker/docker-compose.prod.yml`	`deploy/docker/`	Production cutover compose

In `Work_Credit-RAG_Phase1` (develop branch):

File	Location	Purpose
`Jenkinsfile`	Root	CI/CD pipeline for parent repo
`api/orchestrator.py`	`api/`	

File	Location	Purpose
		Thin wrapper → components/ orchestrator
`api/guardrails.py`	`api/`	Thin wrapper → components/ guardrails
`api/kb.py`	`api/`	Thin wrapper → components/ knowledgebase
`api/tracing.py`	`api/`	Thin wrapper → components/tracing
`dev/patches/*.py`	`dev/patches/`	40 one-off patch scripts
`dev/tests/*.py`	`dev/tests/`	Ad-hoc test scripts
`dev/investigation/*.py`	`dev/investigation/`	fix_*.py, inspect_*.py, update_*.py
`docs/plans/million-user-scale-plan.md`	`docs/plans/`	Scaling plan
`AGENTS.md`	Root	Branch strategy, layer separation
`README.md`	Root	Hierarchy diagram with Mermaid

In `ICS_ChatBot_agent-state` (main branch):

File	Location	Purpose
`README.md`	Root	Agent file location index
`architecture.md`	Root	Service map, context window mgmt
`coding-standards.md`	Root	Python/FastAPI/db/ model-serving/ Persian rules
`git-workflow.md`	Root	PR workflow, branch protection, submodule
`insights.md`	Root	Key decisions, lessons learned
`FILE_MAP.md`	Root	Index of all agent files + onboarding
`onboarding.md`	Root	New-machine setup checklist

File	Location	Purpose
`skills/README.md`	Root	Agent skill definitions per component
`context/context.md`	Root	Current agent mission
`context/todo.md`	Root	Current agent todo
`history/archive/*.md`	`history/archive/`	2 archived agent sessions

16. Acceptance Criteria

#	Criterion	Status
1	All 8 Docker images build successfully from source	⏳ Pending (H200 build)
2	All containers respond to `/health` on test ports	⏳ Pending
3	Open WebUI E2E passes on alt ports (3 sample queries + generic clarify)	⏳ Pending
4	Jenkins pipeline builds, tests, and pushes images to local registry	⏳ Pending (Jenkins install)
5	Per-service cutover to production ports succeeds with health check green	⏳ Pending
6	All 7 services stable on containers for 24 hours	⏳ Pending
7	Rollback tested and documented for at least one service	⏳ Documented

17. Known Gotchas & Risks

Risk	Mitigation
Corporate Squid proxy modifies pip packages → hash failures	Build on H200 where Docker daemon is proxy-configured Keep host venv path; Docker only for H200/privileged hosts

Risk	Mitigation
Vast.ai no Docker → can’t containerize there	
TensorRT-LLM conversion complexity	Fallback to llama.cpp (current) or vLLM
Gemma-4-31B too large for single GPU	Model parallelism (MP=2) across 2× H200 or RTX 3090s
Long context overflow (663KB prompt)	Triton dynamic batching + max context config; app-level truncation
Secrets in `.env` committed accidentally	`.gitignore` blocks `*.env`; pre-commit hook
Port collision during test	Use alt ports for test; only move to prod ports after host process stopped
Host venv broken after cutover	Keep host venvs intact for 1 week as rollback backup
Jenkins job fails mid-deploy	Per-service rollback; Jenkins pipeline has health-check gate

---

Next Steps

1. **Approve this report** — all details documented
2. **Run H200 build** — `bash Work_RAG-Server-Setup/deploy/docker/build-all.sh local-h200:5000`
3. **Alt-port validation** — `docker compose -f docker-compose.test.yml up -d`
4. **E2E test via Open WebUI** — verify full chain on alt ports
5. **Production cutover** — per-service migration to production ports
6. **24h stability verification** — then decommission host venvs

---